

Условный оператор

Python • лекция 2

if

else

and

or

==

СТАРТ

Зачем нужны условия?

Задача

В парк аттракционов приходит человек.

Если его рост меньше 150 см — на аттракцион нельзя.

Программа должна принять решение.

```
height = int(input())
```

```
if height < 150:  
    print("Вам сюда нельзя")
```

Анатомия if

```
if height < 150:  
    print("Вам сюда нельзя")
```

if

«если» — начало
условного
оператора

condition

Условие, которое
даёт True или False

:

После двоеточия
начинается тело
условия

Отступ = часть блока

Команда с отступом выполняется только тогда, когда условие истинно. В Python структура программы задаётся отступами.

Условие всегда превращается в bool

True

Условие истинно

→ тело if выполняется

Пример:
 $170 > 150$

False

Условие ложно

→ тело if пропускается

Пример:
 $140 > 150$

Операторы сравнения

<

меньше

>

больше

<=

меньше или равно

>=

больше или равно

==

равно

!=

не равно

ВАЖНО

= и == — это разные операции

Присваивание: =

Кладём значение в переменную.

```
height = 150  
password = input()
```

Слева — имя переменной.
Справа — новое значение.

Сравнение: ==

Проверяем, одинаковы ли значения.

```
height == 150  
password == "1234"
```

Результат сравнения:
True или False.

Что считается истиной?

Числа

0 → False

любое ненулевое число → True

`bool(0)`

`bool(150)`

Строки

"" → False

непустая строка → True

`bool("")`

`bool("Python")`

Но в if пишем условие

Старайтесь выражать проверку явно:

`if height == 150:`

Так код легче читать и проверять.

Сравнение строк

```
"ABC" < "BAC"  
"AA" < "AB"  
"A" < "AA"
```

Лексикографический порядок

Строки сравниваются последовательно по символам — похоже на порядок слов в словаре.

Сначала сравнивается первый символ. Если он одинаковый — второй, затем третий и т. д.

Важно: фактически Python сравнивает Unicode-коды символов.

Отступы в Python

```
if a < 8:  
    print(a)
```

4 пробела

Стандартный уровень отступа в Python — четыре пробела.

Tab?

Редактор может вставлять четыре пробела при нажатии Tab. Главное — не смешивать табы и пробелы.

Почему это важно

В Python отступ — не просто оформление. Он определяет, какие команды принадлежат условию.

Если условие не выполнено: else

```
password = input()

if password == "1234":
    print("Вы вошли в систему")
else:
    print("Неверный пароль")
```

if

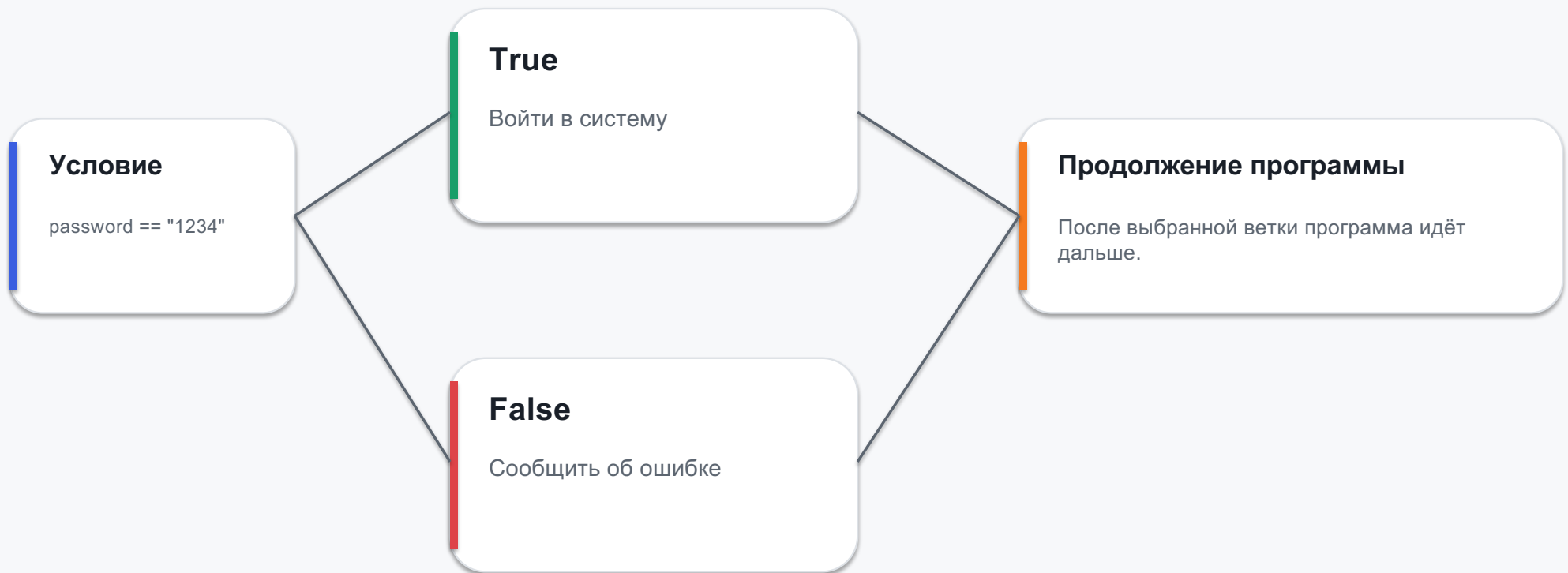
Выполняется, когда условие истинно.

else

Выполняется во всех остальных случаях.

У else нет собственного условия.

Как работает if / else



Больше двух вариантов: elif

```
if height < 60:  
    print("Младенец")  
elif height < 110:  
    print("Ребёнок")  
elif height < 165:  
    print("Подросток")  
else:  
    print("Взрослый")
```

if

Всегда начинает цепочку.

elif

Может быть сколько угодно. Проверяется только если предыдущие ветки не сработали.

else

Необязателен. Может быть только один и только в конце.

Границы в цепочке условий

Каждая следующая ветка уже знает, что предыдущая не выполнялась.

< 60

Младенец

60...109

Ребёнок

110...164

Подросток

≥ 165

Взрослый

Следствие

В `elif height < 110` не обязательно повторять `height >= 60``: до этой ветки программа дошла только потому, что `height < 60`` оказалось `False`.

Составные условия

and

Оба условия должны быть True.

if $a > 0$ and $a < 8$:

Число находится между 0 и 8.

or

Достаточно одного True.

if $a < 0$ or $a > 8$:

Число находится вне диапазона.

not

Инвертирует логическое значение.

if not $a > 8$:

Условие ` $a > 8$ ` должно быть ложным.

and и or: таблица истинности

A	B	A and B	A or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Диапазоны можно писать короче

```
if a > 0 and a < 8:  
    print("Подходит")
```

```
if 0 < a < 8:  
    print("Подходит")
```

то же самое

Python позволяет цепочки сравнений

Запись `0 < a < 8` читается почти как математическая и часто делает код понятнее.

Оператор in

```
"a" in "abc"  
"cat" in "concatenate"  
"7" in number
```

Проверка вхождения

``in`` отвечает на вопрос: содержится ли элемент или подстрока внутри другого объекта?

Результат — True или False.

Для строк

```
`"7" in "1724"` → True
```

Практика 1 • картошка

Условие

Мама печёт картошку.

- масса < 100 г $\rightarrow$ сестре
- $100\text{--}250$ г $\rightarrow$ маме
- больше 250 г $\rightarrow$ папе

Напишите программу, которая определит, кому достанется картошка.

```
mass = float(input())
```

```
if mass < 100:  
    print("Сестре")  
elif mass <= 250:  
    print("Маме")  
else:  
    print("Папе")
```

Практика 1 • проверьте границы

99.9

Сестре

100

Маме

250

Маме

250.1

Папе

Почему тестируем границы?

Большинство ошибок в условиях возникает около ``=``, ``<``, ``<=``, ``>`` и ``>=``. Проверяйте значения прямо на границе и рядом с ней.

Кратность числа

`number % 3 == 0`

Что это значит?

Остаток от деления на 3 равен нулю → число кратно 3.

12 % 3

0 → кратно

14 % 3

2 → не кратно

21 % 7

0 → кратно

Практика 2 • тип данных зависит от условия

```
n = int(input())

if n % 3 == 0:
    a = input()
    b = input()
    print(a + b)
else:
    a = float(input())
    b = float(input())
    print(a * b)
```

Если n кратно 3

`a` и `b` — строки.

"14" + "17" → "1417"

Иначе

`a` и `b` — float.

14.0 * 17.0 → 238.0

Всегда следите за типом данных.

Практика 3 • «ровно одно»

Задача

Вводятся два целых числа.

Если ровно одно из них кратно 4 — вывести:

«4 — моя прелесть»

Как записать условие?

```
if (a % 4 == 0 and b % 4 != 0) or \
    (a % 4 != 0 and b % 4 == 0):
    print("4 — моя прелесть")
```

Идея

Первое кратно, второе — нет
ИЛИ
первое не кратно, второе — да.

Практика 4 • сколько цифр в числе?

Условие

Вводится целое число, не превосходящее 10 000 по модулю.

Определите количество цифр.

Используем только уже изученные операции.

```
if n < 0:  
    n = n * -1  
  
if n < 10:  
    print(1)  
elif n < 100:  
    print(2)  
elif n < 1000:  
    print(3)  
elif n < 10000:  
    print(4)  
else:  
    print(5)
```

Практика 5 • одинаковый рост

Задача

Известен рост трёх человек: `a`, `b`, `c`.

Нужно определить:

- все три одинаковы;
- ровно два одинаковы;
- все различны.

```
if a == b and b == c:  
    print(3)  
elif a == b or b == c or a == c:  
    print(2)  
else:  
    print(1)
```


Поиск максимума из двух

```
a = int(input())  
b = int(input())
```

```
if a > b:  
    print(a)  
else:  
    print(b)
```

Алгоритм

1. Сравниваем два значения.
2. Если `a > b`, максимум — `a`.
3. Иначе максимум — `b`.

Эта идея станет основой более общего алгоритма.

Поиск максимума из пяти

```
m = a  
  
if b > m:  
    m = b  
if c > m:  
    m = c  
if d > m:  
    m = d  
if e > m:  
    m = e  
  
print(m)
```

Главная идея

Храним лучший найденный результат в переменной `m`.

Сначала среди одного числа максимум — это `a`.

Дальше

Каждое новое число сравниваем с текущим максимумом.

Если новое число больше — обновляем `m`.

После последней проверки в `m` лежит максимум.

Тот же алгоритм работает со строками

```
m = a
```

```
if b > m:
```

```
    m = b
```

```
if c > m:
```

```
    m = c
```

```
...
```

Меняется только тип данных

Для чисел `>` сравнивает числовые значения.

Для строк `>` использует лексикографическое сравнение.

Внешне код выглядит одинаково.

Одинаковая операция может работать с разными типами данных.

Вложенные условия

```
line = input()

if line == "дробные числа":
    a = float(input())
    # ... ищем максимум
elif line == "целые числа":
    a = int(input())
    # ... ищем минимум
```

Условие может выбирать алгоритм

В одной ветке программа работает с `float`, в другой — с `int`.

А внутри ветки могут быть новые if

Так появляются вложенные конструкции. Следите за уровнями отступов — они показывают структуру.

Как получить цифры числа?

```
n = 3456
```

```
d = n % 10
```

```
c = n // 10 % 10
```

```
b = n // 100 % 10
```

```
a = n // 1000 % 10
```

% 10

Даёт последнюю цифру числа.

// 10

Удаляет последнюю цифру. Комбинируя `//` и `%`, можно добраться до нужного разряда.

Есть ли в числе цифра 2 или 7?

```
if a == 2 or a == 7 or \
b == 2 or b == 7 or \
c == 2 or c == 7 or \
d == 2 or d == 7:
    print("Да")
```

Способ 1: арифметика

Разбираем число по разрядам и проверяем каждую цифру.

Способ полезен, когда число нужно обрабатывать именно как число.

А можно проще: использовать строку

```
number = input()

if "2" in number or "7" in number:
    print("Да")
```

Способ 2: строки

Если нам важно только наличие символа, число можно не преобразовывать в `int`.

`in` делает проверку намного короче.

Какой способ лучше?

Зависит от задачи. Хороший программист умеет видеть несколько способов решения и выбирать подходящий.

PEP 8 для условий

```
if a<8:  
    print ( a )
```

не надо

```
if a < 8:  
    print(a)
```

лучше

Читаемость — часть качества программы

Пробелы вокруг операторов, четыре пробела для вложенного блока, без пробела перед `:`.